mathematics

MDPI

Article

A Proposal of a Mathematics Problem Generation Tool Using Generative AI for STACK Online Assessment System

Prismahardi Aji Riyantoko [1], Nobuo Funabiki [1,*], Komang Candra Brata [1,2], Noprianto [1], Sischa Wahyuning Tyas [3] and Dwi Arman Prasetya [3]

[1] Department of Information and Communication Systems, Okayama University, Okayama 700-8530, Japan; pnai2m3s@s.okayama-u.ac.jp (P.A.R.); k.candra.brata@ub.ac.id (K.C.B.); noprianto@s.okayama-u.ac.jp (N.)
[2] Department of Informatics Engineering, Universitas Brawijaya, Malang 65145, Indonesia
[3] Department of Data Science, Universitas Pembangunan Nasional Veteran Jawa Timur, Surabaya 60294, Indonesia; sischa_wahyuning.sada@upnjatim.ac.id (S.W.T.); arman.prasetya.sada@upnjatim.ac.id (D.A.P.)
* Correspondence: funabiki@okayama-u.ac.jp

Abstract

The System for Teaching and Assessment using a Computer Algebra Kernel (STACK) is an open source, computer algebra-based online assessment system for teaching and learning mathematics at university. Although the popularity is increasing around the world, its problem generation needs a complex procedure such as algebraic scripting, dynamic randomization, and grading logic, which poses a substantial workload. In this paper, we propose a mathematics problem generation tool using *Generative AI* for STACK. It adopts a *Retrieval-Augmented Generation (RAG)* framework to guide the AI to produce pedagogically aligned problems across *Depth of Knowledge (DoK)* levels, while a *Computer Algebra System (CAS)* validates mathematical precision. The output is rendered into an *XML* template and is imported into the STACK system. For evaluation, we measured the success rate of generating 90 problem files for STACK by the proposal and compared the completion time with their manual generation. *Learning Object Review Instrument (LORI)* was also evaluated for user satisfactions. The results showed that the success rate was 79% while the time was reduced by 35.71%. Furthermore, the *LORI* evaluations demonstrated a feasibility score of 82.1%, confirming the potential to mitigate teacher workload.

Keywords: STACK; Computer Algebra System (CAS); Generative AI; Retrieval-Augmented Generation (RAG); Depth of Knowledge (DoK); Learning Object Review Instrument (LORI)

MSC: 97U50

Academic Editors: Salman Khalid and Aydin Azizi

Received: 31 May 2026
Revised: 27 June 2026
Accepted: 7 July 2026
Published: 9 July 2026

1. Introduction

The System for Teaching and Assessment using a Computer Algebra Kernel (STACK) is an open-source, computer algebra-based question type for that has gained significant prominence in university-level mathematics education due to its capability to deliver formative feedback through algebraically precise question–answer evaluation [1]. The main goal of STACK is to link mathematical problem with a *Computer Algebra System (CAS)* backend, which allows teachers to design questions with dynamic randomization and to automatically evaluate student responses through symbolic computation [2,3]. The use of STACK is particularly beneficial in calculus, linear algebra, and differential equations courses, where precise symbolic reasoning and structured formative feedback are essential

for effective student learning [4,5]. The key accuracy metrics of STACK assessment consist of the mathematical correctness of generated expressions, the logical integrity of grading conditions, and the cognitive alignment of problems with intended learning outcomes [6,7].

To provide interactive and adaptive assessment features, STACK requires to generate problems using a domain-specific scripting environment based on *Maxima*, a powerful open-source *CAS*. This authoring process needs substantial technical proficiency, encompassing algebraic scripting in syntax, dynamic variable randomization, and the construction of multi-node grading logic through [8]. Hence, the time and effort of developing well-structured, pedagogically aligned STACK problems often exceed what mathematics teachers can realistically sustain, particularly for those without a strong programming background [9]. Inaccurate or poorly authored STACK problems could produce inconsistent grading results, reducing the reliability of the assessment and the quality of formative feedback delivered to students [10].

The study by Sangwin highlighted that while STACK provides high flexibility for mathematics assessment, its authoring complexity remains a primary barrier to its widespread adoption among non-specialist teachers [1]. Similarly, researchers involved in large-scale STACK deployments documented that the substantial workload associated with problem creations and maintenance frequently limits the volume and diversity of available problem sets across academic terms [11]. These findings increase the need of accessible tools that can assist novice teachers who generate the STACK-compatible problems without requiring deep technical expertise.

In recent years, *Generative AI* has opened possibilities for automating complex tasks in educational settings [12,13]. Actually, *Large Language Models (LLMs)* have demonstrated remarkable capabilities in generating structured texts, mathematical explanations, and programming codes that can be adapted for automated assessment authoring [14]. However, raw *LLM* outputs are often ungrounded in domain-specific structures and lack alignment with formal pedagogical frameworks, making direct applications to specialized assessment systems such as STACK being unreliable if additional grounding and validation mechanisms are not provided [15,16].

The *Retrieval-Augmented Generation (RAG)* framework has the potential of addressing this limitation by combining domain-specific reference materials with generative capabilities of an *LLM* [17]. By incorporating a curated knowledge base of valid STACK problem templates into a retrieval component, a *RAG*-based system can guide an *LLM* to produce structurally and pedagogically appropriate outputs that conform to the STACK authoring format.

Furthermore, the *Depth of Knowledge (DoK)* framework, which categorizes cognitive demand across hierarchical levels, provides a principled structure for ensuring that generated problems span a meaningful range of difficulty, contributing to more balanced and comprehensive assessment sets [18].

Evaluating the quality of generated STACK problems requires a reliable instrument that can capture multiple dimensions of learning object quality from a teacher's perspective. The *Learning Object Review Instrument (LORI)* was originally developed to assess digital learning objects across multiple quality dimensions including content quality, pedagogical soundness, and acceptability [19]. Adapting a version of *LORI* to the STACK problem domain can provide a comprehensive and validated means of assessing the pedagogical quality and practical acceptability of the generated problems.

Beyond the technical capability of generating STACK problems, teachers' attitudes and motivation are also important factors for the practical adoption of AI-assisted STACK authoring. Prior studies on *Generative AI* in education indicate usefulness that can influence teachers' acceptance of AI-supported tools [20,21]. Therefore, it is important to

examine teachers' initial perceptions of STACK and AI-assisted STACK authoring, since these perceptions may affect whether such tools can be effectively accepted and used in teaching practice.

In this paper, we propose a mathematics problem generation tool using *Generative AI* for STACK. This tool adopts a *RAG* framework implemented with a vector database and *sentence-transformers* embeddings to retrieve relevant STACK templates. The retrieved templates are then used to guide an *LLM*, accessed via *OpenRouter*, in generating problem across *DoK* levels. A *Maxima*-based *CAS* microservice performs multi-seed quality testing on the generated , and a *Jinja2* template engine renders the validated output into STACK-compatible *Moodle XML* format. The tool is designed to reduce teachers' workload in STACK problem authoring while supporting mathematical correctness, pedagogical quality, and teacher-centered validation for effective assessment.

The remainder of this article is organized as follows: Section 2 introduces the related work. Section 3 describes the key technologies adopted in this study. Section 4 presents the design, architecture, and implementation of the proposed tool. Section 5 shows the results and discusses their analysis. Section 6 presents the ablation study. Finally, Section 7 provides conclusions with future work.

2. Related Work

In this section, we provide a literature overview that related to the proposed tool in this paper.

2.1. STACK and Computer Algebra System in Mathematics Assessment

Computer algebra-based online assessment has been an active research area in mathematics education technology. STACK has emerged as the most widely adopted platform for formative assessment of symbolic mathematics.

In ref. [22], Sangwin et al. investigated the design of STACK for the practical online assessment of mathematical proof, demonstrating that *Potential Response Trees (PRTs)* structure can be configured to evaluate free-form symbolic reasoning beyond simple numerical or algebraic equivalence checks. While their work confirmed the expressive power of the STACK authoring framework, the authors emphasized that the complexity of designing valid *PRT* logic for non-routine reasoning tasks substantially increases the technical burden on the problem author, reinforcing the need for automated authoring support tools.

In ref. [23], Kinnear et al. synthesized a collaboratively derived research agenda for e-assessment in undergraduate mathematics, identifying scalability of assessment authoring and pedagogical alignment of digitally generated content as the priority research area. Their agenda highlighted that authoring workload remains a central unresolved challenge in STACK deployment at scale, and called for researches in tools that can assist teachers in creating diverse, well-calibrated problem sets with reduced manual effort, directly motivating the contribution of the present study.

In ref. [24], Weiss et al. reported deployment experiences with computer algebra systems in engineering mathematics courses, documenting the practical impact of *CAS*-based formative assessment on student learning outcomes and teacher satisfactions. However, this study also recorded that teachers consistently reported high time investment in problem authoring as the primary obstacle to expanding their STACK question banks, and that the lack of authoring assistance tools limited the breadth of topics and difficulty levels covered in deployed assessments [25].

2.2. Generative AI in Educational Content Generation

The emergence of *Generative AI*, particularly *LLMs*, has substantially transformed the landscape of automated educational content creation. Several studies have examined the potential and limitations of *LLMs* in generating structured assessment items.

In ref. [26], Kurdi et al. conducted a systematic review of automatic question generation methods for educational purposes, covering rule-based, neural, and transformer-based approaches. The review established that neural approaches significantly outperform earlier rule-based methods in terms of fluency and contextual relevance. However, the authors emphasized that generated questions frequently lack structural consistency and alignment with formal assessment frameworks, highlighting the need for grounding and validation mechanisms in any practical deployment that targets specialized assessment systems.

In ref. [27], Elkins et al. investigated the use of *GPT-4* in generating educational quizzes across undergraduate *STEM* courses using *Bloom's Taxonomy* as a cognitive alignment framework. Their study demonstrated that *LLM*-generated content can achieve quality comparable to human-authored ones in terms of perceived difficulty and discriminability. Unfortunately, the generated outputs were produced in unstructured plain text and were not validated against a *CAS*, rendering them unsuitable for deployment in mathematically rigorous systems such as STACK where algebraic precision and symbolic grading are fundamental requirements [28].

In ref. [29], Pardos et al. examined learning gains of students receiving algebra hints generated by *ChatGPT-4* compared to those generated by human tutors, finding comparable effectiveness in certain problem-solving contexts. However, the study also identified a systematic tendency of *LLM*-generated mathematical contents to exhibit surface-level accuracy while containing subtle symbolic errors that are only detectable through formal algebraic verification [30].This finding directly motivates the *CAS* Validation Stage in the proposed tool, where each generated problem is subjected to multi-seed *Maxima* testing before being made available for teacher download.

2.3. Retrieval-Augmented Generation in Educational Applications

Retrieval-Augmented Generation (RAG) has emerged as a powerful paradigm for improving the factual accuracy and structural grounding of *LLM* outputs in knowledge-intensive generation tasks.

In ref. [31], Siriwardhana et al. extended the *RAG* framework with domain-specific adaptation strategies, demonstrating that a domain-adapted retrieval corpus significantly improves the relevance and correctness of generated content compared to general-purpose retrieval approaches. Their findings support the design choice in our proposed tool of constructing a STACK-specific knowledge base from validated problem templates, rather than relying on general-purpose web-retrieved content during the generation stage.

In ref. [32], Gao et al. conducted a comprehensive survey of *RAG* systems, covering corpus construction, embedding strategies, and generation control mechanisms. The survey identified structured output constraints as a critical design factor when *RAG* is applied to template-driven generation tasks where outputs must conform to a predefined schema. This finding directly informs the architectural separation between *LLM* generation, which produces a structured *JSON* schema, and *XML* rendering, handled entirely by a dedicated *Jinja2* template engine, in the proposed system.

In ref. [33], Asai et al. proposed *Self-RAG* as an adaptive retrieval-augmented framework that learns to retrieve and critically evaluate retrieved passages before generation, demonstrating significant improvements in factual accuracy and structural consistency of outputs. Their analysis of relevance-gated retrieval corroborates the relevance-gated reranking module implemented in the retrieval process of our proposed tool, which applies a

combined cosine similarity and section-aware keyword overlap score to filter low-relevance templates before passing context to the generation prompt [34,35].

2.4. Depth of Knowledge Framework in Assessment Design

The *Depth of Knowledge (DoK)* framework provides a hierarchical taxonomy for categorizing the cognitive complexity of assessment tasks. Its integration into automated content generation represents a meaningful contribution to pedagogically principled assessment design.

In ref. [36], Webb established a foundational *DoK* alignment methodology through large-scale empirical studies of standards and assessment alignment, demonstrating that intentional cognitive level targeting in assessment design leads to significantly more reliable evaluation of student reasoning competencies. The framework's applicability to *STEM* disciplines has been confirmed across multiple subject areas, establishing it as the standard reference taxonomy for cognitively calibrated assessment design in the present study.

In ref. [37], Webb conducted an empirical investigation of *DoK*-aligned assessment in undergraduate mathematics, demonstrating that well-calibrated cognitive level distributions significantly improve assessment validity and the breadth of reasoning competencies measured. The study emphasized that most automated question generation systems fail to maintain intentional *DoK* distributions, resulting in cognitively imbalanced assessment sets that over-represent lower-order recall and under-represent strategic reasoning tasks. This limitation is directly addressed by the *DoK*-guided generation mechanism in the proposed tool.

In ref. [38], Sol et al. examined cognitive complexity in higher education formative assessment design, demonstrating that explicit cognitive demand scaffolding in the assessment authoring process leads to more comprehensive and balanced coverage of intended learning outcomes through experimental evidence. Unfortunately, their implementation relied entirely on manual workflows, reinforcing the importance of developing automated tools that can systematically target specified cognitive levels as part of an end-to-end generation process.

3. Key Technologies

In this section, we introduce key technologies and frameworks adopted in this study. They include the STACK online assessment system, the *Generative AI* and *RAG* framework, and the *Depth of Knowledge (DoK)* framework.

3.1. STACK Online Assessment System

In this paper, we adopted STACK v4.11.1 (STACK Project, University of Edinburgh, Edinburgh, UK) . It is an open-source question-type that can be plugged in to Moodle v5.1 (Moodle Pty Ltd., West Perth, Australia), enabling computer-aided formative assessment of mathematics through a *CAS* backend [39,40]. It allows teachers to define mathematical questions with randomized symbolic parameters, student input validation, and algebraically graded response trees powered by Maxima v5.47.0 (Maxima Project, open-source software)—an open-source *CAS* that serves as the symbolic computation engine for all variable evaluation, answer expression verification, and *PRT* grading logic execution [41].

The STACK authoring workflow requires teachers to declare question variables in *Maxima* syntax, define expected answer expressions, and configure multi-node *PRT* structures that evaluate student responses through a series of algebraic tests and conditional branches [42]. The resulting question is stored in a *Moodle XML*-based format that supports portable import and export across STACK deployments.

3.2. Generative AI and Retrieval-Augmented Generation

Generative AI refers to a class of machine learning models capable of producing novel structured content, including text, code, and data, by learning statistical patterns from large training corpora [43]. In this paper, we adopted *GPT-5.0* (OpenAI, San Francisco, CA, USA) based on the FrontierMath: Benchmarking AI against advanced mathematical research [44], then accessed through the *OpenRouter* API, to serve as the core generative engine. We configured at a temperature of 0.2 to ensure deterministic and structurally consistent outputs [45].

To improve structural and pedagogical grounding, *GPT-5.0* operates within a *RAG* framework built on *Qdrant v1.12.x* (Qdrant Solutions GmbH, Berlin, Germany), a high-performance open-source vector database that manages the embedded representations of curated STACK problem templates [46]. The embedding layer employs the *sentence-transformers/all-MiniLM-L6-v2* model for dense vector representation of both the knowledge base documents and the teacher's generation specification at query time [47].

The entire backend is containerized and deployed using *Docker v4.55.0* (Docker, Inc., Palo Alto, CA, USA), orchestrated with *Docker Compose* to manage the multi-service architecture comprising the *FastAPI* backend, *Qdrant* vector database, and *Maxima* microservice [48]. In *RAG*, a retrieval component identifies the most relevant templates from the knowledge base using vector similarity search, and the *LLM* subsequently conditions its generation on both the teacher's specification and the retrieved context [49].

3.3. Depth of Knowledge Framework

The *Depth of Knowledge (DoK)* framework, developed by Webb, characterizes the cognitive complexity of academic tasks across four hierarchical levels. In the context of university mathematics assessment, *DoK Level 1 (Basic)* corresponds to recall and direct execution of routine procedures. *DoK Level 2 (Intermediate)* demands application of conceptual knowledge and multi-step procedures, and *DoK Level 3 (Advanced)* involves strategic reasoning, justification, and problem solving requiring synthesis across multiple concepts [50].

In the proposed tool, the *DoK* level is an explicit teacher-specified input parameter that constrains both the retrieval stage, by prioritizing templates annotated with the matching cognitive level, and the *LLM* generation prompt, ensuring that generated problems target the intended cognitive demand.

4. Proposal of Mathematics Problem Generation Tool

In this section, we present the proposed mathematics problem generation tool for STACK. It consists of the phases for the system design, the *RAG*-based generation process, the *CAS* validation module, and the *XML* rendering and STACK integration mechanism.

4.1. Separation of LLM Generation and XML Rendering

The critical design insight of this proposal comes from the fundamental observation that raw *LLM* outputs frequently lack the structural conformity required by the STACK *XML* authoring format and the mathematical correctness required for *CAS*-based symbolic grading.

A key architectural principle to address this limitation is the separation of *LLM* generation and *XML* rendering. The *LLM* does not produce STACK *XML* directly. Instead, it generates a structured *JavaScript Object Notation (JSON)* representation, which a dedicated *Jinja2* template engine then renders into the correct *XML* format for each *DoK* level. This separation ensures that structural integrity is maintained by the template layer regardless of generative variation, while the *RAG* retrieval layer grounds the *LLM* output in validated STACK problem templates from the curated knowledge base.

Building on these insights, the proposed tool aims to provide an end-to-end solution for automated STACK problem authoring that substantially reduces teacher workload without compromising assessment quality.

4.2. Three-Stage Generation Process

The proposed tool processes each problem generation request through three stages of *Generation*, *Validation*, and *Rendering* to produce mathematically correct and structurally compatible output with the STACK import mechanism [51]. Figure 1 depicts the mapping concept of the three-stage process and the modules responsible for each stage.

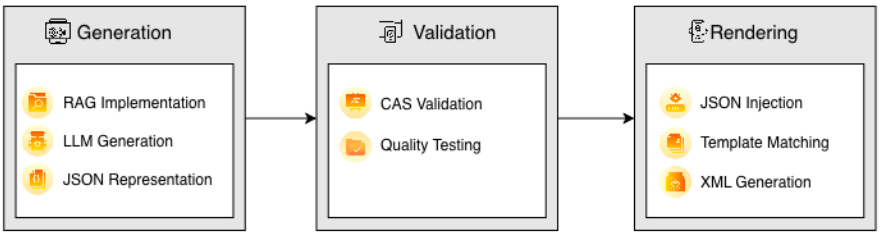


Figure 1. Three-stage generation process of the proposed tool.

The *Generation Stage* encompasses the *RAG Implementation* and *LLM Generation* process, where relevant STACK templates are retrieved from the vector database and used to guide *GPT-5.0* in producing a *JSON Representation* of the problem.

The *Validation Stage* employs the *Maxima* microservice to perform *CAS Validation* and multi-seed *Quality Testing* on the generated specification, verifying mathematical correctness and randomization robustness across multiple parameter instantiations.

The *Rendering Stage* applies *Template Matching* via a *Jinja2* template corresponding to the specified *DoK* level, performing *JSON Injection* to convert the validated *JSON* into a complete STACK-compatible *Moodle XML* file through *XML Generation*.

4.3. Design of Proposed Tool

Figure 2 shows the design of the proposed tool.

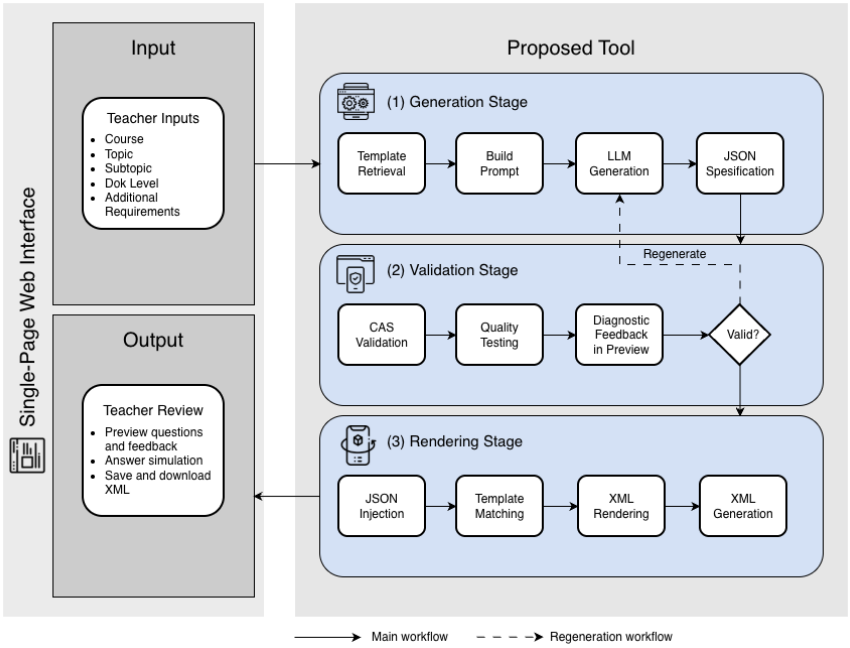


Figure 2. Design overview of the proposed tool.

4.3.1. Input

The input to the tool is the problem specification to be generated, which is submitted by a teacher through a *Single-Page Web Interface*. The input parameters encompass the *Course*, the *Topic*, the *Subtopic*, the *DoK Level*, and *Additional Requirements* for any further constraints. The interface is designed to minimize technical overhead, allowing a teacher to specify the generation requirements without knowledge of *Maxima* scripting or *STACK XML* authoring conventions.

4.3.2. Process

The processing comprises three sequential stages executed by dedicated modules.

In the *Generation Stage*, template retrieval retrieves the most relevant STACK problem templates, which are used to build prompt then submitted to *GPT-5.0* for *LLM* generation, producing a *JSON* specification of the problem.

In the *Validation Stage*, *CAS* validation and quality testing are performed on the generated specification, followed by diagnostic feedback to verify mathematical correctness and randomization robustness; problems failing the validity check are returned for automatic regeneration with structured feedback to the *Generation Stage*.

Upon successful validation, in the *Rendering Stage*, *JSON* injection is applied prior to template matching, followed by *XML* rendering and *XML* generation to convert the validated *JSON* into the final STACK-compatible *Moodle XML* format.

4.3.3. Output

Based on the validated generation result, the teacher review interface allows the teacher to preview questions and feedback, perform answer simulation, and save and download the generated STACK-compatible *XML* file that can be directly imported into a Moodle-hosted STACK deployment.

4.4. System Architecture

The proposed tool integrates *GPT-5.0* via *OpenRouter*, the *Qdrant* vector database, and a containerized *Maxima* microservice into a unified web application deployed through *Docker Compose*. The tool components include a *FastAPI*-based backend server, a *Qdrant* vector database for *RAG* retrieval, an *OpenRouter* API interface for *LLM* generation, a *Maxima* HTTP microservice for *CAS* validation, a *Jinja2* template engine for *XML* rendering, a *SQLite* metrics database for longitudinal performance monitoring, and a single-page web interface.

All external service communications use the *JSON* format. Generated *XML* files are written to a persistent *output_xml* directory. Figure 3 depicts the detailed system architecture of the tool.

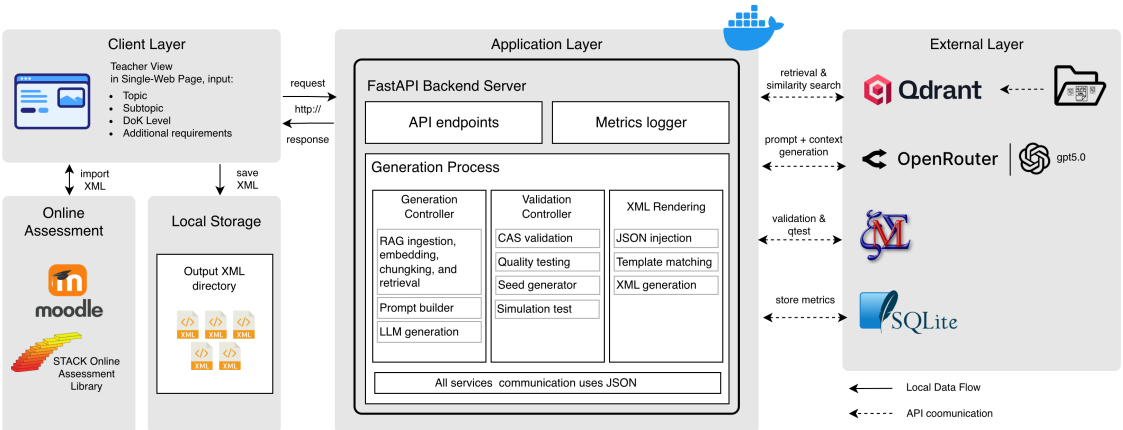


Figure 3. System architecture of the proposed STACK problem generation tool.

The *Qdrant* vector database stores the embedded representations of curated *STACK XML* examples using the *all-MiniLM-L6-v2* embedding model, enabling efficient similarity retrieval during the Generation Stage [52]. At startup, the backend automatically ingests all *STACK XML* examples from the *rag_corpus/xml_examples* directory into the *Qdrant* collection, making the knowledge base immediately available without manual indexing steps. The *Maxima* service runs as a separate containerized HTTP microservice, receiving symbolic computation requests from the backend via REST API, and is monitored by a self-watching process that periodically rescans the corpus directory for newly added templates.

4.5. RAG-Based LLM Generation Module

This module uses the curated *STACK XML* corpus as retrieval-based guidance to assist the *LLM* with generating problem that are aligned with the selected topic.

4.5.1. XML Files in RAG Corpus

The *RAG* corpus is a collection of reference *STACK XML* templates used by the retrieval module in the proposed system. The corpus was prepared from curated *STACK XML* templates for calculus problem generation and is used to provide relevant examples for guiding the *LLM* during problem generation. Table 1 shows the main information of the *RAG* corpus, which consists of 101 curated *STACK XML* templates with a total corpus size of 0.57 MB.

Table 1. The metadata of the RAG corpus.

Component	Description
Corpus directory	rag_corpus/xml_examples
Document format	Curated *STACK XML* templates
Total templates	101 STACK-compatible templates
Total corpus size	0.57 MB
Covered course	Calculus
Covered topics	Limits, Differentiation, Integration
Covered subtopics	Finite Limits, Sequences and Limits, Quotient Rule, Chain Rule, Integration by Parts, u-Substitution
Vector collection	stack_xml_examples
Embedding model	sentence-transformers/all-MiniLM-L6-v2
Indexed metadata	Topic, subtopic, template identifier, and *DoK*-level annotation

Table 2 presents the numbers of templates in the *RAG* corpus by topic, subtopic, and *DoK* level. These numbers are presented to clarify the corpus coverage and to show how the reference templates are organized across the selected calculus topics and difficulty levels. The corpus contains 101 templates from three calculus topics and six subtopics, with 34 Basic, 34 Intermediate, and 33 Advanced templates.

Table 2. Topic, subtopic, and *DoK*-level distribution of the RAG corpus.

Topic	Subtopic	Basic	Intermediate	Advanced	Total
Limits	Finite Limits	5	5	5	15
Limits	Sequences and Limits	6	5	5	16
Differentiation	Quotient Rule	5	5	4	14
Differentiation	Chain Rule	6	6	6	18
Integration	Integration by Parts	5	5	5	15
Integration	u-Substitution	7	8	8	23
Overall	6 subtopics	34	34	33	101

4.5.2. RAG Framework

The *RAG* knowledge base consists of curated *STACK XML* examples covering university-level calculus topics, stored in the *Qdrant* collection *stack_xml_examples*. At the generation time, the teacher's specification, which comprises the course, topic, subtopic, and *DoK* level, is embedded using *all-MiniLM-L6-v2* and used to retrieve the top-*k* most similar problem templates via cosine similarity search.

A section-aware reranking step reorders the retrieved templates by a composite score combining cosine similarity, keyword overlap, and STACK-section boost (higher weights for *questionvariables* and *prt* sections), ensuring that the most structurally and cognitively appropriate examples are passed to the generation prompt.

The retrieved templates and pedagogical context are combined with the teacher specification into a structured prompt that explicitly constrains the *GPT-5.0* output to the *JSON* schema. This schema specifies all required fields: *metadata* (course, topic, subtopic, *DoK* level, tags), *questiontext_latex* (question text with embedded variable references), *questionvariables_maxima* (*Maxima* variable declarations), *inputs* (input type and correct answer expression), *generalfeedback_latex* (general solution), *prt* (*PRT* node configuration with grading tests and feedback), and *seeds* (randomization seed parameters). The *DoK* level is explicitly included in the prompt to guide the structural complexity of the generated *JSON* [53]. The step-by-step workflow of the *RAG*-guided generation is depicted in Figure 4.

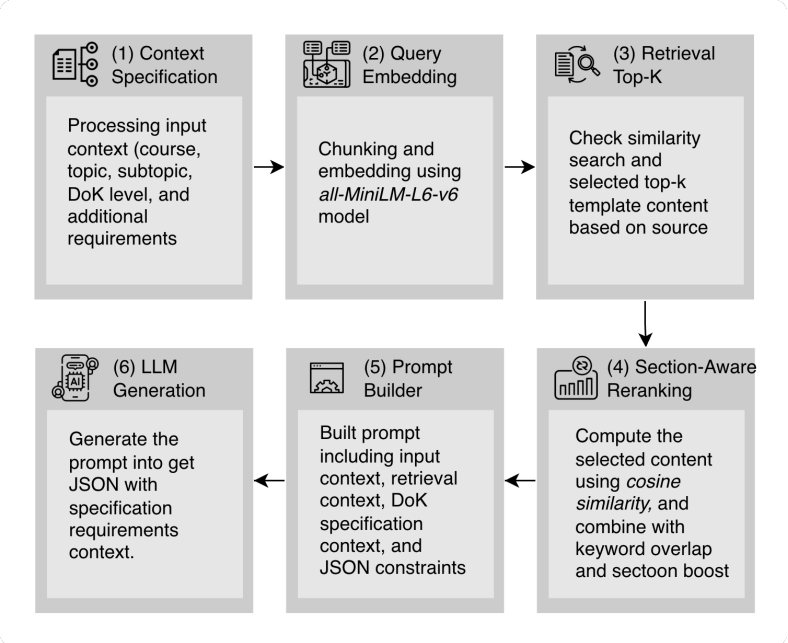


Figure 4. Workflow of the *RAG*-based generation module.

4.6. CAS Validation and Quality Testing Module

The *CAS Validation and Quality Testing Module* employs the *Maxima* to verify the mathematical and functional correctness of each generated *JSON* before *XML* rendering. The validation process consists of two sequential stages: *Seed Preview* and *QTest*.

In the *Seed Preview stage*, a teacher can specify a manual seed value and preview the problem with computed variable values, rendered LaTeX question text, the expected answer evaluated for that seed, and an interactive *XML* preview. The preview interface includes interactive validation capabilities. A teacher can simulate student responses using *Start*, *SaveAnswer*, *FillCorrect*, *Submit*, and *Finish* controls to verify that the *PRT* grading logic produces the expected outcomes for representative correct and incorrect responses.

In *QTest stage*, the tool automatically iterates over multiple seed values to validate the problem's robustness across the randomization space. The minimum number of seeds tested scales with the *DoK* level: 5 seeds for *Basic*, 10 seeds for *Intermediate*, and 20 seeds for *Advanced* problems, reflecting the increasing complexity of the randomization space and grading logic at higher cognitive levels. The *Save XML* function is enabled only when all *QTest* seed validations pass and the *XML* structure is confirmed valid, ensuring that no functionally incorrect problem can reach the output directory.

4.7. XML Rendering Module

Upon successful *QTest* validation, each problem's *JSON* is rendered into the final STACK-compatible *Moodle XML* format using a *DoK*-specific *Jinja2* template. Three dedicated templates are provided:

- *dok_basic.xml.j2*;
- *dok_intermediate.xml.j2*;
- *dok_advanced.xml.j2*.

For each, appropriately structuring the *PRT* configuration and feedback complexity to the corresponding cognitive level is important.

The rendered *XML* file conforms to the *Moodle XML* question format extended with STACK-specific elements including the *questionvariables* block, *questiontext*, *specificfeedback*, and the complete *PRT* node definitions.

The output *XML* file and its companion *JSON* sidecar are saved to the *output_xml* directory with a timestamped filename. The generation metrics for each run are automatically recorded in the *SQLite* metrics database.

4.8. Example of JSON Specification and Rendered STACK XML

The JSON specification generated by the *LLM* contains the variables. Listing 1 depicts this JSON specification as the structured representation of the generated problem. The Jinja2 template engine maps the JSON fields into a prepared STACK *XML* template. Listing 2 depicts the rendered STACK *XML* output produced from the mapped JSON values.

Listing 1. Compact JSON specification generated by the LLM.

```
{
        "metadata": {
                "name": "Evaluate a Finite Limit",
                "course": "Calculus",
                "topic": "Limits",
                "subtopic": "Finite Limits",
                "dok": "Basic"
        },
        "questiontext_latex":
        "Evaluate the limit \\( \\lim_{x \\to 2} (3x+1) \\).",
        "questionvariables_maxima":
        "limit_value: limit(3*x+1, x, 2);",
        "inputs": [
        {
                "name": "ans1",
                "type": "numeric",
                "correct_answer_maxima": "limit_value"
        }
        ],
        "generalfeedback_latex":
        "Substitute \\(x=2\\): \\(3(2)+1=7\\).",
        "prt": {
                "name": "prt1",
                "answer_test": "AlgEquiv",
```

```
                        "sans": "ans1",
                        "tans": "limit_value",
                        "feedback_correct": "Correct.",
                        "feedback_wrong": "Please recheck the substitution."
                }
        }
```

Listing 2. Corresponding STACK XML elements rendered from the JSON specification.

```
        <question type="stack">

        <name>
        <text>Evaluate a Finite Limit</text>
        </name>

        <questiontext format="html">
        <text><![CDATA[
        Evaluate the limit \( \lim_{x \to 2} (3x+1) \).
        [[input:ans1]] [[validation:ans1]]
        ]]></text>
        </questiontext>

        <questionvariables>
        <text>limit_value:  limit(3*x+1, x, 2);</text>
        </questionvariables>

        <input>
        <name>ans1</name>
        <type>numeric</type>
        <tans>limit_value</tans>
        </input>

        <generalfeedback format="html">
        <text><![CDATA[Substitute \(x=2\):  \(3(2)+1=7\).]]></text>
        </generalfeedback>

        <prt>
        <name>prt1</name>
        <node>
        <answertest>AlgEquiv</answertest>
        <sans>ans1</sans>
        <tans>limit_value</tans>
        <truefeedback><text>Correct.</text></truefeedback>
        <falsefeedback><text>Please recheck the substitution.</text></falsefeedback>
        </node>
        </prt>

        </question>
```

4.9. User Interface

The user interface of the proposed tool provides an intuitive single-page web interface for the complete problem generation workflow. When a teacher opens the application, the left panel presents the generation specification controls: *Course* selector, *Topic* and *Subtopic* drop-down menus, *DoK* difficulty selector, and an optional free-text requirements field. Figure 5 depicts the user interface examples.

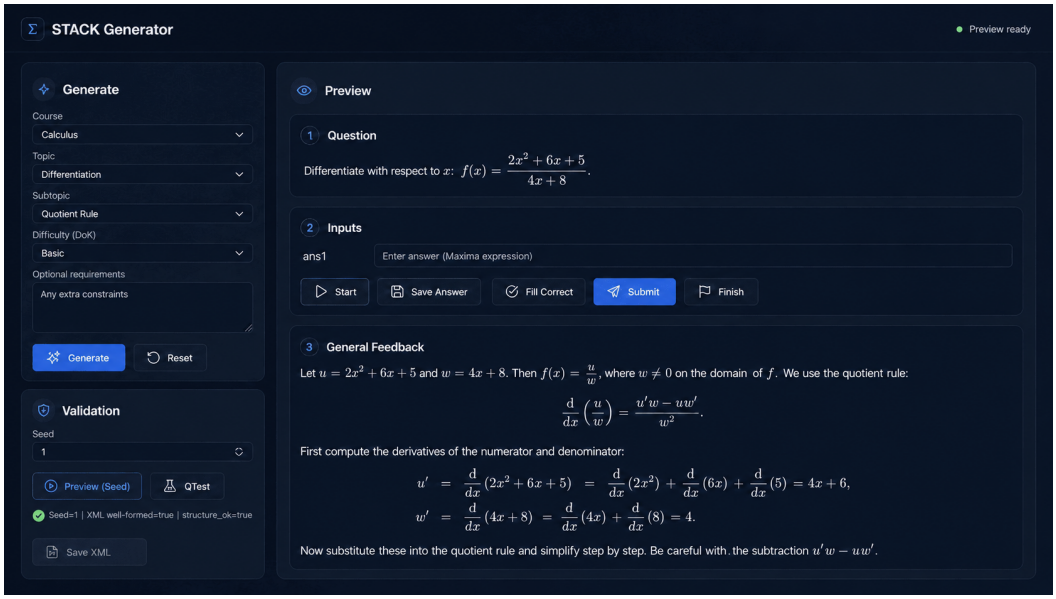
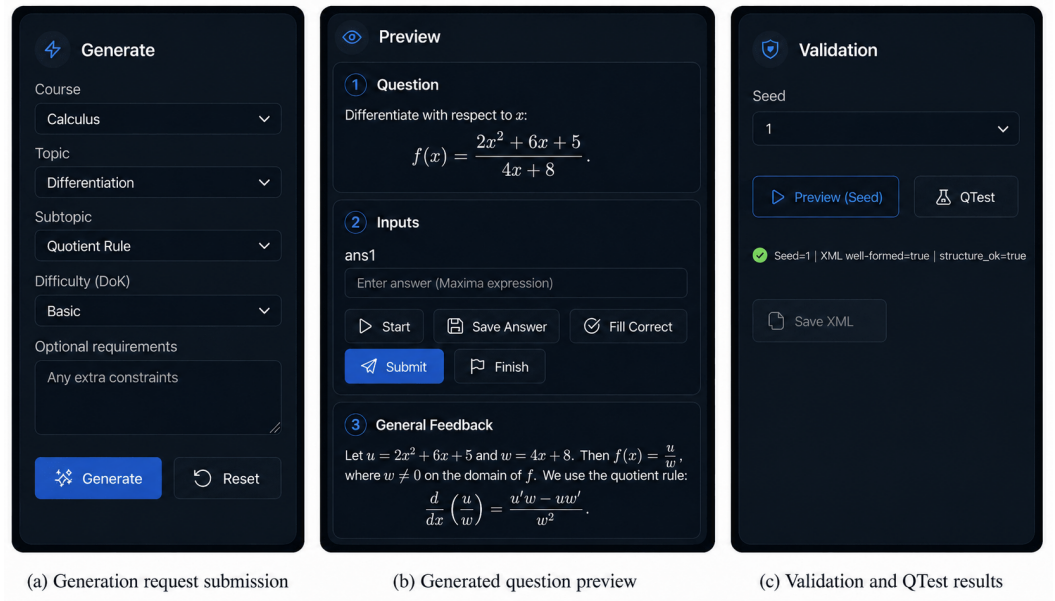


Figure 5. User interface for STACK problem generation.

Upon submitting a generation request (Figure 6a), the tool processes the *RAG* retrieval and *GPT-5.0* generation in the background and presents a preview panel displaying the rendered LaTeX question text, the configured student input areas, and the general worked solution feedback (Figure 6b). The validation section enables the teacher to perform seed-based preview and automated *QTest*, with a real-time status indicator and validation results summary (Figure 6c). The *Save XML* button becomes active only after a successful *QTest*, ensuring that the complete workflow from specification to output is validated before saving.



(a) Generation request submission (b) Generated question preview (c) Validation and QTest results

Figure 6. Interface details of the proposed tool: (**a**) generation form, (**b**) question preview, and (**c**) *QTest* validation.

4.10. Teacher-Driven Design of Problem Generation

Teachers can define the required through the web-application interface before generating a problem. They select the *course*, *topic*, *subtopic*, and difficulty level to specify the general context of the problem. After completing these required fields, teachers can use the *Optional Requirements* field to provide additional that are not captured by the predefined options. These additional may include the expected mathematical structure, parameter

constraints, cognitive demand, answer format, feedback direction, and instruction style, as shown in Table 3.

Table 3. Examples of inputs for the *Optional Requirements* field.

Requirement Type	Example of Teacher Input
Mathematical structure	Generate a definite-integral problem involving the product of a linear term and an exponential term, solvable by integration by parts.
Parameter constraint	Use randomized positive integer coefficients within a small range and avoid values that produce overly large expressions.
Cognitive demand	Require students to determine the appropriate integration method before performing the calculation.
Answer format	Require the final answer in exact symbolic form and do not accept decimal approximations.
Feedback direction	Provide step-by-step feedback that explains why integration by parts is used and how each part of the equation is applied.
Instruction style	Use concise instructions that clearly state the task and expected answer format without unnecessary context.

5. System Evaluations

In this section, we evaluate the proposed tool. This tool is assessed through generation success rate measurement, completion time comparison with manual authoring, and user satisfaction evaluation using the *LORI* instrument.

5.1. Testing Environment

The testing environment for this evaluation is designed to assess the proposed tool in practical STACK problem authoring scenarios representative of university-level calculus instruction. The evaluation covered the *Calculus* course, which is the primary subject domain supported by the current prototype. In this course, three topics, *Limits*, *Differentiation*, and *Integration*, with two subtopics for each were selected:

- *Limits* encompasses *Finite Limits* and *Sequences and Limits*.
- *Differentiation* encompasses *Quotient Rule* and *Chain Rule*.
- *Integration* encompasses *Integration by Parts* and *u-Substitution*.

For each of the six subtopics, five problems were generated for each of the three *DoK* levels (*Basic*, *Intermediate*, and *Advanced*), yielding a total target of 90 problem files across all subtopics and difficulty levels. We chose these subtopics based on their coverage in standard university calculus syllabi and their representativeness of varying symbolic complexity in STACK problem authoring.

In this evaluation, we did not compare the performance of different *LLM* models or embedding configurations. The goal of this study was to analyze the performance of the *RAG*-guided generation process for STACK problem authoring using a representative *LLM* and to compare its efficiency and quality against the established manual workflow.

5.2. Testing Two Conditions

We compared the proposed tool against manual STACK problem authoring across the same 90 problem to demonstrate the efficiency.

5.2.1. Manual STACK Authoring Condition

The manual authoring condition represents the conventional STACK problem creation workflow using *STACK 4.11.1* on *Moodle 5.1*. Five mathematics teachers that have STACK authoring experience individually completed the problem generation tasks using the standard STACK question editor. In this workflow, each teacher navigated to the STACK question type in the question bank, wrote *Maxima 5.47* variable declarations in the *Question variables* field, composed the question text with embedded *CAS* variable references, defined the expected answer expression, configured *PRT* nodes with algebraic tests and feedback messages, and verified the question's behavior using STACK's built-in testing tools. The total completion time required to complete each problem was recorded for comparison.

5.2.2. Proposed Tool Generation Condition

The five teachers used the proposed tool to complete the generation tasks through the four sequential steps for each problem:

1. Selecting the *Course*, *Topic*, *Subtopic*, and *DoK* level in the left panel and submitting the generation request;
2. Reviewing the preview with an optional custom seed;
3. Running the automated *QTest* to validate across multiple seeds;
4. Downloading the *XML* output file for STACK import.

The time from a specification submission to an *XML* download was recorded for comparison with the manual operation.

5.3. Testing Process

To assess the accuracy and efficiency of the proposed tool, a comparison was conducted between the manual STACK authoring and the proposed tool generation. In the manual, each teacher completed the assigned problems over multiple sessions to avoid fatigue effects. In the proposed tool, the same teachers used the tool on different days to minimize carry-over learning effects.

The generated *XML* files from the proposed tool were imported into a *STACK 4.11.1* deployment on *Moodle 5.1*, where each problem was verified in correct rendering and grading behavior. The technical performance metrics for each generation run. They include *RAG* quality metrics, *DoK* alignment scores, *Maxima* performance data, and *XML* quality indicators, which were automatically captured by the system's metrics monitoring infrastructure and stored in the *SQLite* metrics database for analysis.

5.4. Baseline Methods Comparison

The baseline methods are the reference methods used to compare the performance of the proposed tool with basic generation approaches. This baseline methods help this study to clarify how each added component contributes to the quality of the generated *STACK XML* output.

We conducted the baseline method comparison involving three AI-based generation methods: *GPT* only, *GPT* with prompt engineering, and *GPT* with *RAG*. The baseline methods were designed to represent different levels of AI support for generating *STACK XML* output, as shown in Table 4.

Each baseline method generated 90 problems. The generated outputs were compared with the proposed tool using four metrics: *DoK* alignment, *CAS* validity, *XML* well-formedness, and STACK import success rate. The four metrics were used to evaluate the quality of the generated *STACK XML* output.

Table 4. Configuration and objective of each baseline method.

Baseline Method	Configuration	Objective
GPT only	The method generates problems using a basic prompt from the teacher.	The method measures the basic performance of the *LLM* in producing *STACK XML* output.
GPT with Prompt Engineering	The method generates problems using few-shot examples and step-by-step prompt instructions.	The method measures the effect of prompt engineering in producing *STACK XML* output.
GPT with *RAG*	The method generates problems using a structured prompt and retrieved STACK examples.	The method measures the effect of retrieved STACK examples in producing *STACK XML* output.

Table 5 presents the performance comparison results between the proposed tool and the baseline methods. The results show that the proposed tool achieved the highest scores across all metrics, particularly in *XML* well-formedness and STACK import success rate. This finding highlights the feasibility of the proposed tool for generating valid and importable *STACK XML* output.

Table 5. Performance comparison of the proposed tool and baseline methods across evaluation metrics.

Method	Metric Performance			
	***DoK* Alignment**	***CAS* Validity**	***XML* Well-Formedness**	**STACK Import Success (%)**
GPT only	0.06	0.11	0.11	6%
GPT with Prompt Engineering	0.17	0.23	0.11	9%
GPT with *RAG*	0.33	0.40	0.38	42%
Proposed Tool	0.82	0.84	0.99	79%

5.5. Technical Performance Evaluation

To investigate the performance of the proposed tool, we analyzed the metrics recorded during the 90 generation runs across the evaluation period.

5.5.1. Performance Metrics

The performance of the proposed tool was evaluated in four categories [54]. The *LLM and RAG Generation* category covers *Faithfulness* (degree to which generated output adheres to retrieved context), *Answer Relevance* (alignment with teacher specification), *Answer Correctness* (mathematical accuracy), and *Answer Similarity* (semantic similarity to reference problems), while *RAG Retrieval* category covers *Context Precision* and *Context Recall* [55].

The *DoK Alignment* category covers *Level Consistency* (degree to which generated cognitive demand matches the specified *DoK* level) and *Scale of Alignment* (binary indicator of correct *DoK* classification). The *CAS* category covers *Maxima* performance validity. The *XML* covers *Well-formedness*, *Exhaustivity*, and *Specificity*. The overall *STACK Import Success Rate* serves as the primary composite performance indicator.

5.5.2. Technical Performance Results

Table 6 shows the performance metrics across the 90 generation runs. The *LLM and RAG* generation metrics indicate reliable generative behavior, with an *Answer Relevance* of 0.73, reflecting strong alignment between generated problem content and teacher .

The *Answer Correctness* of 0.65 reflects the remaining challenge of generating mathematically precise content from language model outputs alone, underscoring the indispensable role of the subsequent *CAS* validation stage.

The *RAG Retrieval* metrics show moderate *Context Precision* (0.55) and *Context Recall* (0.56), indicating that the current *RAG* corpus provides relevant but not exhaustive coverage for all specified subtopics.

The *DoK* alignment results confirm a *Level Consistency* of 0.82 and a perfect *Scale of Alignment* of 1.00.

The *XML Quality* metrics demonstrate near-perfect *Well-formedness* (0.99) and *Exhaustivity* (1.00), confirming that the *Jinja2* template rendering reliably produces structurally complete and schema-valid *STACK XML* files.

The *XML Specificity* score of 0.40 indicates that there remains room for improvement in the level of detail in the generated *PRT* feedback messages.

Table 6. Evaluation metrics of the proposed tool.

Component	Category	Metric	Score
LLM & RAG	Generation	Faithfulness	0.73
		Answer Relevance	0.74
		Answer Correctness	0.65
	Retrieval	Context Precision	0.55
		Context Recall	0.56
DoK	Alignment	Level Consistency	0.82
		Scale of Alignment	1.00
CAS	Performance	Validity	0.84
XML	Structure & Content	Well-formedness	0.99
		Exhaustivity	1.00
		Specificity	0.40

Table 7 presents representative errors observed in the generated problems that affected *Answer Correctness*. The errors involved algebraic inconsistency, invalid *Maxima* syntax, variable mismatch, randomization edge cases, and *PRT* answer mismatch. This finding explains why some generated problems had acceptable *XML* structure but still failed in mathematical validation.

Table 7. Representative error types and their effects on *Answer Correctness*.

Error Type	Example	Effect
Algebraic inconsistency	The expected answer did not match the required derivative or integral.	Incorrect answer key.
Invalid *Maxima* syntax	The expression used notation that was not executable in *Maxima*.	*CAS* validation failed.
Variable mismatch	A variable appeared in the answer or feedback but was not declared.	Expression evaluation failed.
Randomization issue	Some seed values produced division by zero or undefined expressions.	*QTest* failed for certain seeds.
PRT answer mismatch	The *PRT* used an inconsistent expected answer or answer test.	Correct answers could be rejected or wrong answers accepted.

5.5.3. Success Rate Analysis

Table 8 presents the *STACK Import Success Rate* by *DoK* level. The proposed tool achieved an overall *STACK Import Success Rate* of 79% (71 out of 90 problems), demonstrating practical feasibility for automated STACK problem generation. The success rate was highest for *Basic* problems (26 out of 30, 86.7%) and progressively lower for *Intermediate* (23 out of 30, 76.7%) and *Advanced* (22 out of 30, 73.3%) problems.

Table 8. STACK import success rate by *DoK* level.

DoK Level	Target	Success (%)	Failure (%)	Success Import Rate (%)
Basic	30	26	4	86.7
Intermediate	30	23	7	76.7
Advanced	30	22	8	73.3
Overall	90	71 (79%)	19 (21%)	–

The consistent pattern of decreasing success rate with increasing *DoK* level reflects the higher *PRT* configuration complexity and the more demanding *Maxima* randomization logic required for higher cognitive-level problems. The 21% overall failures predominantly arise from *Maxima* validation failures due to edge cases in the randomization space for complex expressions, rather than structural *XML* errors, as evidenced by the near-perfect *XML Well-formedness* score.

5.5.4. Failure Case Analysis

The failed cases refer to the problem files that did not complete the validation or import process. To examine these issues, we analyzed the system's error information and grouped the cases into six error types: XML Rendering, Moodle/STACK Import, Maxima Incorrect, Randomization Issue, PRT Issue, and Invalid Content. These categories represent the errors that occurred and provide an initial indication of how the failures appeared in the dataset.

Table 9 presents the 19 failed cases by *DoK* level and failure category. The results show that failures were more frequent in Maxima expressions, *PRT* configuration, and randomization issues than in *XML* rendering and Moodle/STACK import. This result indicates that the remaining failures were related to mathematical validation and assessment logic.

Table 9. Summary of failed cases by *DoK* level and failure category.

DoK Level	Failure Categories						Total
	XML Rendering	Moodle/STACK Import	Maxima Incorrect	Random. Issue	*PRT* Issue	Invalid Content	
Basic	1	1	1	0	0	1	4
Intermediate	1	0	2	1	2	1	7
Advanced	1	1	2	2	2	0	8
Overall	3	2	5	3	4	2	19

5.6. Completion Time Comparison

To assess the efficiency of the proposed tool, we compared the total completion time for generating STACK problems using the proposed tool against the manual authoring condition.

5.6.1. Completion Time Metric

The *Completion Time Reduction Rate* (R_{time}) is calculated as follows:

$$R_{\text{time}} = \frac{T_{\text{manual}} - T_{\text{tool}}}{T_{\text{manual}}} \times 100\%. \quad (1)$$

where T_{manual} and T_{tool} are the mean completion times (in minutes) per problem under the manual and proposed tool conditions, respectively.

5.6.2. Completion Time Comparison Results

Table 10 presents the completion time comparison between the manual STACK authoring and the proposed tool across the three *DoK* levels. The proposed tool reduced the mean completion time at all levels by 20.98%, 35.71%, and 50.42% for *Basic*, *Intermediate*, and *Advanced* problems, respectively. The reductions were statistically significant and showed large effect sizes, indicating that the proposed tool improved authoring efficiency, especially as problem complexity increased. Overall, the proposed tool achieved an average completion time reduction of 35.70% across all *DoK* levels.

Table 10. Completion time comparison by *DoK* level.

***DoK* Level**	**Manual STACK**	**Proposed Tool**	**Reduction**	***p*-Value**	**Effect Size**
	Mean ± SD (min)	**Mean ± SD (min)**			
Basic	11.84 ± 1.45	9.36 ± 1.41	20.98%	<0.025	1.73
Intermediate	22.56 ± 3.33	14.50 ± 1.88	35.71%	<0.003	2.98
Advanced	37.40 ± 3.42	18.54 ± 2.72	50.42%	<0.001	6.10
Overall	23.93 ± 2.73	14.13 ± 2.00	35.70%	–	–

5.7. Learning Object Review Instrument

To evaluate the pedagogical quality and usability of the generated STACK problems from the teacher's perspective, a domain-adapted modification of *LORI* was developed [56]. This instrument comprises 25 items organized across five dimensions: *Relevance* (Items 1–5), *Appropriateness* (Items 6–10), *Reliability* (Items 11–15), *Pedagogy Quality* (Items 16–20), and *Acceptability* (Items 21–25). Each item is rated on a five-point Likert scale, where 1 represents the lowest quality and 5 represents the highest quality [57]. Table 11 presents all 25 items of the *LORI* instrument.

The *Feasibility Score (FS)* for each dimension and for the overall instrument is computed by:

$$FS = \frac{\bar{S}}{S_{\max}} \times 100\%. \quad (2)$$

where $\bar{S}$ is the overall score across all teachers and items within the dimension, and $S_{\max} = 5$ is the maximum possible item score. Before deployment, the instrument underwent content validity assessment using the *Scale-level Content Validity Index (S-CVI/Ave)* and internal reliability assessment using *Cronbach's Alpha*, following established psychometric validation procedures [58].

Table 11. *LORI* instrument for STACK generation problem quality evaluation.

Dimension	No.	Item Statement
A—Relevance	1	The question accurately represents the requested topic and subtopic.
	2	The question effectively measures the targeted learning objective.
	3	The question covers the material with ideal depth, neither too shallow nor too complex.
	4	The question respects students' prerequisite knowledge and avoids untested assumptions.
	5	The mathematical definitions and terminology used are academically accurate.
B—Appropriateness	6	The difficulty level is appropriate for the target student profile.
	7	The numbers generated by the random variables are reasonable and meaningful.
	8	The estimated time required to solve the question is realistic for an actual exam scenario.
	9	The question is free from unnecessary information or confusing distractions.
	10	The problem context is logical and relatable to students.
C—Reliability	11	The answer key and solution steps are free from mathematical errors.
	12	The random variables are robust and do not cause errors, such as division by zero.
	13	The *XML* code imports into the STACK question bank and runs seamlessly.
	14	The question correctly handles input syntax, such as distinguishing between a syntax error and a wrong answer.
	15	The variable naming within the internal STACK code is clean and organized.
D—Pedagogical Quality	16	The question text is clear, unambiguous, and easy to understand.
	17	The specific feedback provided for incorrect answers is educationally meaningful.
	18	The full worked solution in the general feedback is structured logically and sequentially.
	19	The question clearly instructs students on how to format their answer, such as rounding, units, or variable names.
	20	The language tone is natural, polite, and encouraging.
E—Acceptability	21	The question is ready for classroom use without further editing.
	22	Using these drafts is more helpful than creating a question manually from scratch.
	23	The question template has strong potential for repeated use in the future.
	24	The code is easy to integrate into existing quizzes.
	25	I have full confidence in the quality of the output generated by this tool.

5.8. Expert Evaluation

The expert evaluation assessed the quality of the generated STACK problems by administering a validated *LORI* instrument to five university mathematics teachers who directly reviewed the generated problems.

5.8.1. Instrument Validation, Teacher Profiles, and Procedure

Prior to the expert evaluation, the 25-item *LORI* instrument was validated in content validity and internal reliability. Five expert reviewers independently assessed the relevance of each instrument item using a four-point content validity scale [59].

The *S-CVI/Ave* was 0.912, confirming excellent content validity and strong expert consensus that each item is highly relevant for assessing the quality of STACK-generated problems. The consistency of the instrument was assessed using *Cronbach's Alpha*, yielding a value of 0.778 and indicating strong internal reliability and stable response patterns across teachers [60,61].

Table 12 presents the profiles of the five expert teachers who participated in the *LORI* assessment. All teachers were university mathematics teachers in Indonesian higher education institutions with experiences involving STACK on . Each teacher reviewed a representative sample of generated problems covering all three topics and three *DoK* levels on a *Moodle 5.1* test instance where the generated STACK problems had been imported, allowing direct interaction with questions and grading behavior.

Table 12. Teacher profiles in the *LORI* assessment.

Teacher	Teaching Exp.	Field of Expertise	STACK Proficiency (1–5)
T1	>5 years	Applied Mathematics	3
T2	<5 years	Linear Algebra	3
T3	<5 years	Mathematical Modeling	3
T4	>5 years	Operations Research	2
T5	<5 years	Computational Mathematics	2

Note: STACK proficiency scale: 1 = beginner, 3 = intermediate, and 5 = expert. All teachers consented to participate in this research.

The participating teachers were intentionally selected from users with limited prior STACK proficiency because the proposed tool is designed to support teachers who are not yet highly experienced in STACK question authoring. Therefore, the teacher evaluation in this study primarily reflects the perspective of novice-to-intermediate STACK users.

Their responses were collected using an online form, after each teacher completed the instrument immediately in their hands-on review session to minimize recall bias [62].

5.8.2. Expert Evaluation Results

Table 13 presents the *LORI* results across all five dimensions and five teachers. The overall feasibility score of 82.1% confirms a satisfactory level of quality across all evaluated dimensions, indicating that the generated STACK problems are suitable for classroom use from the perspective of experienced mathematics teachers when using this proposed tool approach.

The *Relevance* (80.0%) dimension received the lowest feasibility score, primarily due to teacher ratings on the Item 3 and Item 4. This finding suggests that the *RAG* knowledge base and *DoK* specification mechanisms should be further refined to more precisely align with the assumed prerequisite background for each subtopic.

The *Appropriateness* (82.0%) and *Reliability* (82.4%) dimensions received comparable scores, confirming that the *DoK*-guided generation produces problems with suitable difficulty calibration and that the *Maxima*-validated outputs are mathematically correct and functionally robust in the STACK environment.

The *Pedagogy Quality* (83.2%) dimension received the highest feasibility score, confirming that the generated problems exhibit appropriate language clarity, educationally meaningful feedback messages, and logically structured worked solutions.

The *Acceptability* (83.0%) dimension received a high score, reflecting that teachers found the generated problems highly useful for reducing authoring workload, suitable for reuse in future assessments, and straightforward to integrate into existing quiz environments.

Table 13. *LORI* results by teachers.

Dimension	LORI Score (/5)	Feasibility Score (%)
A—Relevance	4.00	80.0
B—Appropriateness	4.10	82.0
C—Reliability	4.12	82.4
D—Pedagogy Quality	4.16	83.2
E—Acceptability	4.15	83.0
Overall	4.11	82.1

5.9. Teacher Attitude and Motivation Using STACK and AI-STACK

This section examines teachers' attitudes and motivation toward STACK and AI-assisted STACK problem authoring. It describes the adapted questionnaire, scoring procedure, and descriptive acceptance results obtained from the participating teachers.

5.9.1. Attitude and Motivation Questionnaire

Teachers' attitudes and motivation were evaluated using a 5-point Likert questionnaire adapted from the *Teachers' Acceptance of AI in Education (TAAI)* instrument [63]. Table 14 presents the adapted questionnaire items, which are grouped into two dimensions: STACK Usage and STACK-AI Acceptance. These dimensions were used to examine teachers' perceptions of using STACK and their acceptance of AI-assisted STACK authoring.

Table 14. Questionnaire items adapted for teacher attitude and motivation toward STACK usage and STACK-AI acceptance.

Dimension	No.	Item Statement
STACK Usage	1	I use or intend to use STACK to support mathematics assessment more actively.
	2	I am motivated to use STACK because it can provide automatic grading and feedback for students.
	3	I consider STACK useful for creating varied and reusable mathematics questions.
	4	The technical complexity of STACK limits the intensity of my use in regular teaching.
	5	I would use STACK more frequently if the authoring process became easier and faster.
STACK-AI Acceptance	6	I am interested in using *Generative AI* to support STACK problem authoring.
	7	I am motivated to use this tool to generate STACK questions more actively.
	8	I believe this tool can reduce the technical burden of STACK authoring.
	9	I would consider using this tool regularly if *CAS* validation and teacher review are included.
	10	I feel confident using AI-assisted STACK authoring when the generated problems can be verified before classroom use.

The questionnaire was completed by five teachers with experience of using STACK and the proposed tool. The responses were summarized using descriptive statistics, including mean scores, agreement rates, and qualitative interpretations. The mean score for each item was calculated using Equation (3) [63].

$$\bar{x}_j = \frac{1}{N} \sum_{i=1}^{N} x_{ij}, \tag{3}$$

where $\bar{x}_j$ is the mean score of item j, x_{ij} is the score given by teacher i, and $N = 5$ is the number of participating teachers.

The agreement rate was defined as the percentage of teachers who selected 4 or 5, representing agree and strongly agree responses. It was calculated using Equation (4) [63].

$$\text{Agreement Rate}_j = \frac{n_j(x \geq 4)}{N} \times 100\%, \tag{4}$$

where $n_j(x \geq 4)$ is the number of teachers who selected 4 or 5 for item j. The qualitative interpretation of teacher acceptance was derived from the mean score of each questionnaire item, as summarized in Table 15.

Table 15. Interpretation criteria for teacher acceptance scores.

Mean Score Range	Interpretation
1.00–1.80	Very low acceptance
1.81–2.60	Low acceptance
2.61–3.40	Moderate acceptance
3.41–4.20	High acceptance
4.21–5.00	Very high acceptance

5.9.2. Teacher's Acceptance Analysis

Table 16 presents the teacher acceptance results for STACK usage and *STACK-AI* acceptance. The STACK Usage dimension had a mean score of 3.85 with a 76.8% agreement rate, while the *STACK-AI* acceptance dimension had a higher mean score of 4.08 with an 81.6% agreement rate. Both dimensions were interpreted as high acceptance. These results suggest that the participating teachers were motivated to use STACK and positively accepted AI-assisted STACK authoring when teacher review and validation were included in the workflow.

Table 16. Descriptive results of teachers' attitude and motivation toward STACK and *STACK-AI* use.

Dimension	Mean Score	Agreement (%)	Interpretation
STACK Usage	3.84	76.8%	High acceptance
STACK-AI Acceptance	4.08	81.6%	High acceptance

5.10. Discussion

Measuring the generation success rate, completion time efficiency, and pedagogical quality in an expert perspective are essential aspects of evaluating the feasibility of this automated STACK problem generation tool, given the direct relationship between these metrics and the practical utility of the tool for mathematics teachers. An interesting finding in the evaluation results is the trade-off between *DoK* level and generation success rate. While the proposed tool offers substantial time savings across all *DoK* levels, the success rate decreases as problem complexity increases, reflecting the greater structural and algebraic demands that *Advanced* problems impose on both the Generation Stage and the Validation Stage.

The 79% overall success rate confirms that the *RAG*-guided generation and *CAS* validation process can reliably produce STACK-compatible problems in the majority of cases, while the 21% failure rate at initial generation indicates that the regeneration mechanism with *Maxima* diagnostic feedback is a necessary component of the workflow.

The completion time comparison shows a clear practical efficiency gain, with time reduction rates of 20.98%, 35.71%, and 50.42% for *Basic*, *Intermediate*, and *Advanced* problems, respectively. The increasing reduction rate across the *DoK* levels indicates that the benefit of the proposed tool becomes more evident as the authoring task becomes more complex and requires technical effort. However, this reduction reflects lower initial authoring effort rather than full automation, because teachers must still review, refine, and check the generated problems in *STACK Moodle* before assessment use.

The *LORI* feasibility score of 82.1%, combined with the instrument's strong content validity (*S-CVI/Ave*: 0.912) and internal reliability (α: 0.778), confirms that the proposed tool delivers pedagogically sound outputs that are suitable for classroom deployment without extensive manual post-editing.

Since the research in this paper is still at the preliminary stage, the current implementation covers the *Calculus* course across three topic areas and six subtopics. This targeted scope allows a controlled and rigorous evaluation while providing a validated foundation for extension to additional courses and topics. The relatively lower *XML Specificity* score (0.40), together with the *Relevance* dimension's comparatively lower *LORI* score, points to clear directions for improvement: enriching *PRT* feedback content generation through stronger pedagogical grounding in the *RAG* corpus, expanding retrieval coverage for topic-specific exemplars, and refining *DoK*-level prerequisite in the generation prompts to better constrain the assumed prior knowledge scope.

However, the number of participated teachers was small, and most of them had only novice-to-intermediate experience in STACK problem authoring. Therefore, the findings

may not fully generalize to teachers with more advanced STACK proficiency. Future work should involve a larger and more representative teacher group.

6. Ablation Study

The ablation study evaluates how component-level configurations influence the proposed tool. The three selected components represent key controllable parts of the workflow: retrieval quality, teacher additional requirements, and *DoK*-based cognitive guidance. For each configuration, 90 problems were generated using the same problem specification to ensure a consistent comparison across all components.

The retrieval and reranking configurations compare how many retrieved *STACK XML* templates are used as generation references after similarity ranking, where the top-1, top-3, and top-5 reranking configurations refer to the number of highest-ranked *STACK XML* templates retrieved and provided to the *LLM* as reference examples. For additional requirements to teachers, the configurations compare three conditions in the *Optional Requirements* field: no additional requirements means that the field was left empty, mixed additional requirements means that the field contained selected teacher such as mathematical structure, parameter constraints, feedback direction, or answer format, and detailed pedagogical requirements means that the field contained more complete instructional constraints. For *DoK* prompt specification, default guidance means that the prompt used only the selected *DoK* level, moderate guidance means that the prompt included a brief description of the expected cognitive demand, and comprehensive guidance means that the prompt included explicit *DoK*-level criteria for guiding the generated problem.

Table 17 presents the component-level results of the selected configurations, with the overall score calculated from *RAG*-Answer Relevance, *RAG*-Answer Correctness, *DoK* Alignment, and *CAS* Validity. For the embedding model and reranking component, *all-MiniLM-L6-v2* with reranking top-3 achieved the highest overall score of 0.76, indicating that using the three highest-ranked retrieved templates produced the best result in this configuration set. For additional requirements to teachers, detailed pedagogical requirements achieved the highest overall score of 0.81, while mixed additional requirements were marked as the selected configuration because they better represent practical teacher input, which can vary in detail and structure. For *DoK* prompt specification, comprehensive *DoK* prompt guidance achieved the highest overall score of 0.76, indicating that more explicit *DoK* guidance improved the overall generation performance.

Table 17. Component-level performance results for selected configurations of the proposed tool.

Components	Configuration	RAG-Answer Relevance	RAG-Answer Correctness	DoK Alignment	CAS Validity	Overall
Embedding model and reranking	*all-MiniLM-L6-v2* + reranking top-3 *	0.74	0.65	0.82	0.84	0.76
	all-MiniLM-L6-v2 + reranking top-5	0.83	0.56	0.76	0.73	0.72
	all-MiniLM-L6-v2 + reranking top-1	0.55	0.42	0.62	0.58	0.54
Teacher additional requirements and specification	Detailed pedagogical requirements	0.86	0.73	0.89	0.77	0.81
	Mixed additional requirements *	0.74	0.65	0.82	0.84	0.76
	No additional requirements	0.54	0.45	0.64	0.69	0.58
DoK prompt specification	Comprehensive *DoK* prompt guidance *	0.74	0.65	0.82	0.84	0.76
	Moderate *DoK* prompt guidance	0.69	0.57	0.72	0.78	0.69
	Default *DoK* prompt guidance	0.61	0.48	0.54	0.70	0.58

7. Conclusions

This paper proposed a STACK mathematics problem generation tool using *Generative AI* by integrating *RAG*-guided generation, *DoK*-based problem specification, *Maxima*-based *CAS* validation, and *Jinja2*-based *XML* rendering. The evaluation results showed that the proposed tool can generate STACK-compatible calculus problems with a satisfactory import success rate, particularly for lower and intermediate difficulty levels, while higher *DoK* problems still require more robust validation due to their greater algebraic and grading complexity. The completion time comparison with the manual generation indicated that the tool can reduce the authoring workload of mathematics teachers, with the strongest benefit observed in more complex problem authoring scenarios. In addition, the expert evaluation using the *LORI* instrument confirmed that the generated problems were pedagogically acceptable, practically useful, and suitable as reusable drafts for classroom assessment. Since this work is preliminary and limited to calculus topics, our future work will focus on enriching the *RAG* corpus, improving *PRT* feedback specificity, strengthening the *Maxima*-based regeneration mechanism, and extending the evaluation to broader mathematics courses and teacher populations.

Author Contributions: Conceptualization, P.A.R. and N.F.; methodology, P.A.R. and N.F.; application tools, P.A.R. and N.; visualization, P.A.R. and K.C.B.; implementation, P.A.R., D.A.P. and S.W.T.; writing—original draft, P.A.R.; writing—review and editing, N.F.; supervision, N.F. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Ethical review and approval were waived by the Ethics Committee for Social Sciences and Humanities of the Badan Riset dan Inovasi Nasional (BRIN) Regulation No. 22 of 2022 because human participants were involved in evaluating the generated mathematics problems to validate their feasibility and assess the efficiency of the proposed tool. This study involved voluntary participation by five university lecturers, did not collect sensitive personal data, and all collected data were anonymized.

Informed Consent Statement: Informed consent was obtained from all participants involved in the study. Participants were informed of the study objectives, the voluntary nature of their participation, and the anonymization of all performance data prior to analysis. Written informed consent was obtained before the evaluation session commenced.

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding author.

Acknowledgments: We would like to thank all the colleagues in the Distributing System Laboratory at Okayama University and lecturers at Malang University, Surabaya University, Lamongan Islamic University, UIN Walisongo Semarang, and UPN Veteran Jakarta who participated in this study.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Sangwin, C.J. *Computer Aided Assessment of Mathematics*; Oxford University Press: Oxford, UK, 2013. https://doi.org/10.1093/acprof:oso/9780199660353.001.0001.
2. Rasila, A.; Harjula, M.; Zenger, K. Automatic Assessment of Mathematics Exercises: Experiences and Future Prospects. In *ReekTori 2007: Symposium of Engineering Education*; Teaching and Learning Development Unit, Helsinki University of Technology: Espoo, Finland, 2007; pp. 70–80.
3. Lowe, T.W.; Mestel, B.D. Using STACK to Support Student Learning at Masters Level: A Case Study. *Teach. Math. Its Appl.* **2020**, *39*, 61–70. https://doi.org/10.1093/teamat/hrz001.
4. Nakamura, Y.; Nakahara, T.; Kaneko, M.; Takato, S. Authoring Quizzes with Interactive Content on the Mathematics e-Learning System STACK. In *Computational Science and Its Applications—ICCSA 2017*; Lecture Notes in Computer Science; Springer: Cham, Switzerland, 2017; Volume 10407, pp. 273–284. https://doi.org/10.1007/978-3-319-62401-3_21.

5. Harjula, M.; Malinen, J.; Rasila, A. STACK with State. *MSOR Connect.* **2017**, *15*, 60–69. https://doi.org/10.21100/msor.v15i2.408.
6. Sangwin, C.J.; Grove, M. STACK: Addressing the Needs of the Neglected Learners. In *Proceedings of the Web Advanced Learning Conference and Exhibition, WebALT*; Oy WebALT Inc., University of Helsinki: Helsinki, Finland, 2006; pp. 81–96.
7. Majander, H.; Rasila, A. Experiences of Continuous Formative Assessment in Engineering Mathematics. In *Tutkimus suuntaamassa 2010-luvun matemaattisten aineiden opetusta*; Tampereen yliopistopaino Oy–Juvenes Print: Tampere, Finland, 2011; pp. 197–214.
8. Lawson, D.; Grove, M.; Croft, T. The Evolution of Mathematics Support: A Literature Review. *Int. J. Math. Educ. Sci. Technol.* **2020**, *51*, 1224–1254. https://doi.org/10.1080/0020739X.2019.1662120.
9. Hattie, J.; Timperley, H. The Power of Feedback. *Rev. Educ. Res.* **2007**, *77*, 81–112. https://doi.org/10.3102/003465430298487.
10. Black, P.; Wiliam, D. Assessment and Classroom Learning. *Assess. Educ. Princ. Policy Pract.* **2006**, *5*, 7–74. https://doi.org/10.1080/0969595980050102.
11. Zerva, K.; Sangwin, C.J.; Jones, I.; Quinn, D. Rejuvenating the HELM Workbooks as Online STACK Quizzes in 2020. *Int. J. Emerg. Technol. Learn.* **2022**, *17*, 69–88. https://doi.org/10.3991/ijet.v17i23.35923.
12. Bommasani, R.; Hudson, D.A.; Adeli, E.; Altman, R.; Arora, S.; von Arx, S.; Bernstein, M.S.; Bohg, J.; Bosselut, A.; Brunskill, E.; et al. On the Opportunities and Risks of Foundation Models. *arXiv* **2021**, arXiv:2108.07258. https://doi.org/10.48550/arXiv.2108.07258.
13. Holmes, W.; Bialik, M.; Fadel, C. *Artificial Intelligence in Education: Promise and Implications for Teaching and Learning*; Center for Curriculum Redesign: Boston, MA, USA, 2019; ISBN 978-1794293700.
14. Brown, T.B.; Mann, B.; Ryder, N.; Subbiah, M.; Kaplan, J.; Dhariwal, P.; Neelakantan, A.;Shyam, P.; Sastry, G.; Askell, A.; et al. Language Models Are Few-Shot Learners. In *Advances in Neural Information Processing Systems*; Curran Associates: Red Hook, NY, USA, 2020; Volume 33, pp. 1877–1901.
15. Ji, Z.; Lee, N.; Frieske, R.; Yu, T.; Su, D.; Xu, Y.; Ishii, E.; Bang, Y.J.; Madotto, A.; Fung, P. Survey of Hallucination in Natural Language Generation. *ACM Comput. Surv.* **2023**, *55*, 248. https://doi.org/10.1145/3571730.
16. Frieder, S.; Pinchetti, L.; Chevalier, A.; Griffiths, R.-R.; Salvatori, T.; Lukasiewicz, T.; Petersen, P.C.; Berner, J. Mathematical Capabilities of ChatGPT. In *NeurIPS 2023 Datasets and Benchmarks*; Curran Associates Inc.: Red Hook, NY, USA, 2023.
17. Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Küttler, H.; Lewis, M.; Yih, W.T.; Rocktäschel, T.; et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*; Curran Associates: Red Hook, NY, USA, 2020; Volume 33, pp. 9459–9474.
18. Webb, N.L. *Depth of Knowledge Levels for Four Content Areas*; Wisconsin Center for Education Research: Madison, WI, USA, 2002.
19. Nesbit, J.; Belfer, K.; Leacock, T. *Learning Object Review Instrument (LORI) User Manual*; E-Learning Research and Assessment Network: Vancouver, BC, Canada, 2003.
20. Lu, H.; He, L.; Yu, H.; Pan, T.; Fu, K. A Study on Teachers' Willingness to Use Generative AI Technology and Its Influencing Factors: Based on an Integrated Model. *Sustainability* **2024**, *16*, 7216. https://doi.org/10.3390/su16167216.
21. Cabellos, B.; de Aldama, C.; Pozo, J.-I. University Teachers' Beliefs about the Use of Generative Artificial Intelligence for Teaching and Learning. *Front. Psychol.* **2024**, *15*, 1468900. https://doi.org/10.3389/fpsyg.2024.1468900.
22. Bickerton, R.T.; Sangwin, C.J. Practical Online Assessment of Mathematical Proof. *Int. J. Math. Educ. Sci. Technol.* **2021**, *53*, 2637–2660. https://doi.org/10.1080/0020739X.2021.1896813.
23. Kinnear, G.; Jones, I.; Sangwin, C.; Alarfaj, M.; Davies, B.; Fearn, S.; Foster, C.; Heck, A.; Henderson, K.; Hunt, T.; et al. A Collaboratively-Derived Research Agenda for E-Assessment in Undergraduate Mathematics. *Int. J. Res. Undergrad. Math.* **2024**, *10*, 201–231. https://doi.org/10.1007/s40753-022-00189-6.
24. Weiss, V.; Tobin, P. Use of Calculators with Computer Algebra Systems in Test Assessment in Engineering Mathematics. *ANZIAM J.* **2016**, *57*, C51–C65. https://doi.org/10.21914/anziamj.v57i0.10445.
25. Zeynivandnezhad, F.; Emilio Fernández, R.; binti Mohammad Yusof, Y.; binti Ismail, Z. Fostering Mathematical Thinking through a Computer Algebra System in a Differential Equation Course. *Int. Electron. J. Math. Educ.* **2025**, *20*, em0826. https://doi.org/10.29333/iejme/16078.
26. Kurdi, G.; Leo, J.; Parsia, B.; Sattler, U.; Al-Emari, S. A Systematic Review of Automatic Question Generation for Educational Purposes. *Int. J. Artif. Intell. Educ.* **2020**, *30*, 121–204. https://doi.org/10.1007/s40593-019-00186-y.
27. Elkins, S.; Kochmar, E.; Serban, I.V.; Bhambhoria, J. How Teachers Can Use Large Language Models and Bloom's Taxonomy to Create Educational Quizzes. In *Proceedings of the 38th AAAI Conference on Artificial Intelligence*; AAAI: Washington, DC, USA, 2024; pp. 23053–23059.
28. Brown, N.; South, K.; Venkatasubramanian, S.; Wiese, E.S. Designing Ethically-Integrated Assignments: It's Harder Than It Looks. In *Proceedings of the 2023 ACM Conference on International Computing Education Research–Volume 1*; Association for Computing Machinery: New York, NY, USA, 2023; pp. 177–190. https://doi.org/10.1145/3568813.3600126.
29. Pardos, Z.A.; Bhandari, S. Learning Gain Differences Between ChatGPT and Human Tutor Generated Algebra Hints. *arXiv* **2023**, arXiv:2302.06871. https://doi.org/10.48550/arXiv.2302.06871.

30. Hendrycks, D.; Burns, C.; Kadavath, S.; Arora, A.; Basart, S.; Tang, E.; Song, D.; Steinhardt, J. Measuring Mathematical Problem Solving with the MATH Dataset. In *Advances in Neural Information Processing Systems*; Curran Associates: Red Hook, NY, USA, 2021; Volume 34.
31. Siriwardhana, S.; Weerasekara, R.; Wen, E.; Hu, T.; Bhashitha, T.; Muthalagu, R.; Deng, S. Improving the Domain Adaptation of Retrieval Augmented Generation Models for Open Domain Question Answering. *Trans. Assoc. Comput. Linguist.* **2023**, *11*, 1–17. https://doi.org/10.1162/tacl_a_00530.
32. Gao, Y.; Xiong, Y.; Gao, X.; Jia, K.; Pan, J.; Bi, Y.; Dai, Y.; Sun, J.; Wang, M.; Wang, H. Retrieval-Augmented Generation for Large Language Models: A Survey. *arXiv* **2024**, arXiv:2312.10997. https://doi.org/10.48550/arXiv.2312.10997.
33. Asai, A.; Wu, Z.; Wang, Y.; Sil, A.; Hajishirzi, H. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. In *Proceedings of the International Conference on Learning Representations*; OpenReview: San Diego, CA, USA, 2024. Available online: https://openreview.net/forum?id=hSyW5go0v8 (accessed on 25 February 2026).
34. Izacard, G.; Grave, E. Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering. In *Proceedings of EACL 2021*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2021; pp. 874–880. https://doi.org/10.18653/v1/2021.eacl-main.74.
35. Zhao, P.; Zhang, H.; Yu, Q.; Wang, Z.; Geng, Y.; Fu, F.; Yang, L.; Zhang, W.; Jiang, J.; Cui, B. Retrieval-Augmented Generation for AI-Generated Content: A Survey. *arXiv* **2024**, arXiv:2402.19473. https://doi.org/10.48550/arXiv.2402.19473.
36. Webb, N.L. Alignment of Science and Mathematics Standards and Assessments in Four States. *NISE Res. Monogr.* **1999**, *18*, 1–128.
37. Webb, N.L. Issues Related to Judging the Alignment of Curriculum Standards and Assessments. *Appl. Meas. Educ.* **2007**, *20*, 7–25. https://doi.org/10.1080/08957340709336728.
38. Sol, K.; Sok, S.; Heng, K. Rethinking Assessment in Higher Education in the Age of Generative AI. In *Encyclopedia of Educational Innovation*; Peters, M.A., Heraud, R., Eds.; Springer: Singapore, 2025; pp. 1–5. https://doi.org/10.1007/978-981-13-2262-4_327-1.
39. Moodle. Moodle 5.1 Release Notes. Moodle Developer Resources. 2025. Available online: https://moodledev.io/general/releases/5.1 (accessed on 1 March 2026).
40. Sangwin, C.J.; Harjula, M.; Rasila, A. STACK Documentation and Architecture. STACK Project. 2024. Available online: https://docs.stack-assessment.org (accessed on 1 March 2026).
41. Maxima. Maxima 5.47 Computer Algebra System. Maxima Project. 2023. Available online: https://maxima.sourceforge.io (accessed on 1 March 2026).
42. Ellonen, O.; Vesapuisto, M.; Vekara, T. Experiences on Development and Design of STACK Problems for Circuit Analysis. *Athens J. Technol. Eng.* **2020**, *7*, 185–204. https://doi.org/10.30958/ajte.7-3-2
43. Zhao, W.X.; Zhou, K.; Li, J.; Tang, T.; Wang, X.; Hou, Y.; Min, Y.; Zhang, B.; Zhang, J.; Dong, Z.; et al. A Survey of Large Language Models. *arXiv* **2023**, arXiv:2303.18223. https://doi.org/10.48550/arXiv.2303.18223.
44. Glazer, E.; Erdil, E.; Besiroglu, T.; Chiou, D.; Ho, B.; Petrov, A.; Plastiras, C.; Atkinson, K.; Adrià, F.-L.; Codreanu, M.; et al. FrontierMath: A Benchmark for Evaluating Advanced Mathematical Reasoning in AI. *arXiv* **2024**, arXiv:2411.04872. https://doi.org/10.48550/arXiv.2411.04872.
45. OpenAI. *GPT-5 System Card*; OpenAI: San Francisco, CA, USA, 2025. Available online: https://openai.com/index/gpt-5-system-card/ (accessed on 1 March 2026).
46. Qdrant. Qdrant v1.12.x Vector Database Documentation. Qdrant. 2024. Available online: https://qdrant.tech/documentation (accessed on 1 March 2026).
47. Reimers, N.; Gurevych, I. Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks. In *Proceedings of EMNLP-IJCNLP 2019*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2019; pp. 3982–3992. https://doi.org/10.18653/v1/D19-1410.
48. Docker Inc. Docker Desktop 4.55.0 Release Notes. Docker Documentation. 2025. Available online: https://docs.docker.com/desktop/release-notes (accessed on 1 March 2026).
49. Karpukhin, V.; Oğuz, B.; Min, S.; Lewis, P.; Wu, L.; Edunov, S.; Chen, D.; Yih, W. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of EMNLP 2020*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2020; pp. 6769–6781. https://doi.org/10.18653/v1/2020.emnlp-main.550.
50. Krathwohl, D.R. A Revision of Bloom's Taxonomy: An Overview. *Theory Pract.* **2002**, *41*, 212–218. https://doi.org/10.1207/s15430421tip4104_2.
51. Ouyang, L.; Wu, J.; Jiang, X.; Almeida, D.; Wainwright, C.L.; Mishkin, P.; Zhang, C.; Agarwal, S.; Slama, K.; Ray, A.; et al. Training Language Models to Follow Instructions with Human Feedback. In *Advances in Neural Information Processing Systems*; Curran Associates: Red Hook, NY, USA, 2022; Volume 35, pp. 27730–27744.
52. Devlin, J.; Chang, M.W.; Lee, K.; Toutanova, K. BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of NAACL-HLT 2019*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2019; pp. 4171–4186. https://doi.org/10.18653/v1/N19-1423.

53. Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Xia, F.; Chi, E.; Le, Q.V.; Zhou, D. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems*; Curran Associates: Red Hook, NY, USA, 2022; Volume 35, pp. 24824–24837. https://doi.org/10.52202/068431-1800.
54. Yang, X.; Sun, K.; Xin, H.; Sun, Y.; Bhatt, N.; Chen, X.; Choudhary, S.; Gui, R.D.; Jiang, Z.W.; Jiang, Z.; et al. CRAG—Comprehensive RAG Benchmark. *arXiv* **2024**, arXiv:2406.04744. https://doi.org/10.48550/arXiv.2406.04744.
55. Es, S.; James, J.; Espinosa-Anke, L.; Schockaert, S. RAGAS: Automated Evaluation of Retrieval Augmented Generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2024; pp. 150–163. https://doi.org/10.18653/v1/2024.eacl-demo.16.
56. Leacock, T.L.; Nesbit, J.C. A Framework for Evaluating the Quality of Multimedia Learning Resources. *J. Educ. Technol. Soc.* **2007**, *10*, 44–59.
57. Boone, H.N.; Boone, D.A. Analyzing Likert Data. *J. Ext.* **2012**, *50*, 48. https://doi.org/10.34068/joe.50.02.48.
58. Polit, D.F.; Beck, C.T.; Owen, S.V. Is the CVI an Acceptable Indicator of Content Validity? Appraisal and Recommendations. *Res. Nurs. Health* **2007**, *30*, 459–467. https://doi.org/10.1002/nur.20199.
59. Lynn, M.R. Determination and Quantification of Content Validity. *Nurs. Res.* **1986**, *35*, 382–385. https://doi.org/10.1097/00006199-198611000-00017.
60. Taber, K.S. The Use of Cronbach's Alpha When Developing and Reporting Research Instruments in Science Education. *Res. Sci. Educ.* **2018**, *48*, 1273–1296. https://doi.org/10.1007/s11165-016-9602-2.
61. Cronbach, L.J. Coefficient Alpha and the Internal Structure of Tests. *Psychometrika* **1951**, *16*, 297–334. https://doi.org/10.1007/BF02310555.
62. Podsakoff, P.M.; MacKenzie, S.B.; Lee, J.-Y.; Podsakoff, N.P. Common Method Biases in Behavioral Research: A Critical Review of the Literature and Recommended Remedies. *J. Appl. Psychol.* **2003**, *88*, 879–903. https://doi.org/10.1037/0021-9010.88.5.879.
63. Tang, Y.; Zhong, L. K-12 Teachers' Adoption of Generative AI for Teaching: An Extended TAM Perspective. *Educ. Sci.* **2026**, *16*, 136. https://doi.org/10.3390/educsci16010136.